PROCESAMIENTO DE LENGUAJE NATURAL

JUAN MANUEL HURTADO ANGULO
MANUEL ALBERTO GONZÁLEZ GONZÁLEZ
WILLIAM ALBERTO REINA GARCÍA

Profesor: Mg. LUIS EDUARDO FERRO DIEZ

Universidad ICESI

Facultad de Ingeniería
Maestría en Inteligencia Artificial Aplicada

Santiago de Cali, Colombia

Transformer para Análisis de Sentimientos en Textos Turísticos

1. Introducción

El análisis automático de sentimientos es una tarea central dentro del procesamiento de lenguaje natural (NLP). En este trabajo se implementa una arquitectura basada en Transformers para clasificar reseñas turísticas en español según su polaridad.

2. Problema de investigación

Las reseñas de usuarios contienen información valiosa sobre la percepción de destinos turísticos. Sin embargo, analizar manualmente grandes volúmenes de texto resulta costoso.

Por ello se plantea el uso de modelos de aprendizaje profundo capaces de identificar automáticamente el sentimiento expresado en cada reseña.

3. Dataset utilizado

El dataset utilizado corresponde al conjunto 'alexcom/analisis-sentimientos-textos-turisticos-mx-polaridad' disponible en Hugging Face. Este dataset contiene reseñas turísticas en español con una etiqueta de sentimiento en escala de 1 a 5.

4. Fundamentos teóricos

La arquitectura Transformer fue introducida por Vaswani et al. (2017) y se basa en mecanismos de atención que permiten modelar dependencias entre palabras dentro de una secuencia sin utilizar redes recurrentes.

El mecanismo de atención se define como:

$$\text{Attention}(Q,K,V) = \text{softmax}((QK^T)/\text{sqrt}(d_k))\ V$$

5. Arquitectura del modelo

El modelo implementado utiliza el TransformerEncoder de PyTorch e incluye los siguientes componentes:

- Token Embedding
- Positional Encoding
- Multi-Head Self-Attention
- Feed-Forward Network
- Clasificador final

Estudiantes: Juan M Hurtado A - Manuel A González G - William A Reina G

6. Configuración experimental

Se realizaron experimentos variando diferentes hiperparámetros del modelo con el objetivo de analizar su impacto en el desempeño.
Variables evaluadas:

- Tipo de tokenizador
- Tamaño del embedding
- Positional encoding (sinusoidal vs learnable)
- Número de cabezas de atención
- Parámetro beta del mecanismo de atención

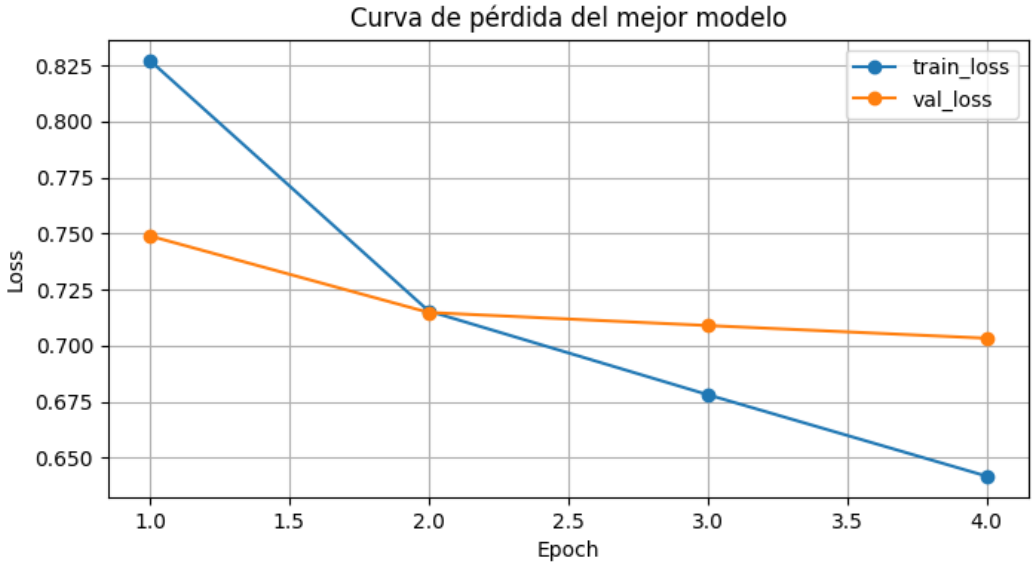
7. Resultados experimentales

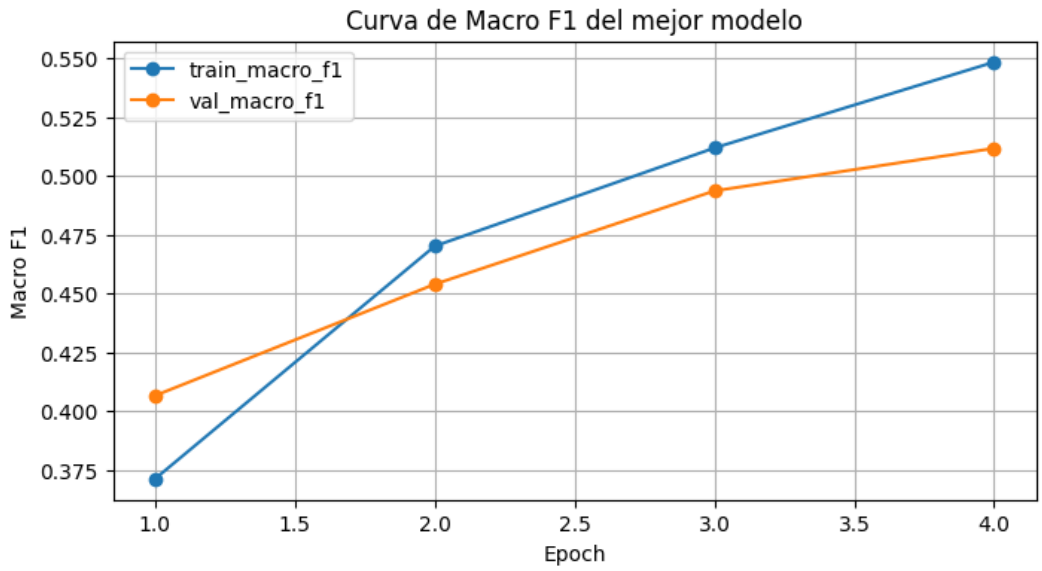
{'epoch': 1, 'train_loss': 0.8689187410199818, 'train_acc': 0.6513873418799174, 'train_macro_f1': 0.3395462410402795, 'val_loss': 0.7487549202520055, 'val_acc': 0.6761450706623532, 'val_macro_f1': 0.41433017252763554}
{'epoch': 2, 'train_loss': 0.7258519533002552, 'train_acc': 0.6903861570878236, 'train_macro_f1': 0.4570104904883598, 'val_loss': 0.7306626245460579, 'val_acc': 0.6914694364038821, 'val_macro_f1': 0.44506981914636407}
{'epoch': 3, 'train_loss': 0.6889381898653765, 'train_acc': 0.7043482579299483, 'train_macro_f1': 0.49850475596489885, 'val_loss': 0.7105913810742962, 'val_acc': 0.6977126965208014, 'val_macro_f1': 0.47059040675224323}
{'epoch': 4, 'train_loss': 0.6561529946110711, 'train_acc': 0.719424205231531, 'train_macro_f1': 0.5330502145558665, 'val_loss': 0.7138644156676671, 'val_acc': 0.6973721550598786, 'val_macro_f1': 0.48933885852682524}
{'epoch': 1, 'train_loss': 0.8593453444362904, 'train_acc': 0.6486346512667343, 'train_macro_f1': 0.3434668233070482, 'val_loss': 0.7528432338294013, 'val_acc': 0.6800612974629661, 'val_macro_f1': 0.40987167130684854}
{'epoch': 2, 'train_loss': 0.7270387924150495, 'train_acc': 0.6902016984384866, 'train_macro_f1': 0.4569126335901914, 'val_loss': 0.723763294693779, 'val_acc': 0.6907883534820365, 'val_macro_f1': 0.4619462598716124}
{'epoch': 3, 'train_loss': 0.6887815546434665, 'train_acc': 0.7056394684753073, 'train_macro_f1': 0.49683651094555054, 'val_loss': 0.720340684407852, 'val_acc': 0.6944207957318803, 'val_macro_f1': 0.4701887548577185}
{'epoch': 4, 'train_loss': 0.6542446515549325, 'train_acc': 0.7198569736011294, 'train_macro_f1': 0.5305616719959501, 'val_loss': 0.7121544037345533, 'val_acc': 0.6973153981497248, 'val_macro_f1': 0.4900484200314045}
{'epoch': 1, 'train_loss': 0.8589696891326775, 'train_acc': 0.6465204713627947, 'train_macro_f1': 0.3440231038013417, 'val_loss': 0.7456858846647986, 'val_acc': 0.6808558942051195, 'val_macro_f1': 0.4266172455680254}
{'epoch': 2, 'train_loss': 0.7289792433296511, 'train_acc': 0.6877115066724369, 'train_macro_f1': 0.45024582702476207, 'val_loss': 0.7230878511812207, 'val_acc': 0.6912424087632669, 'val_macro_f1': 0.4534198544490583}

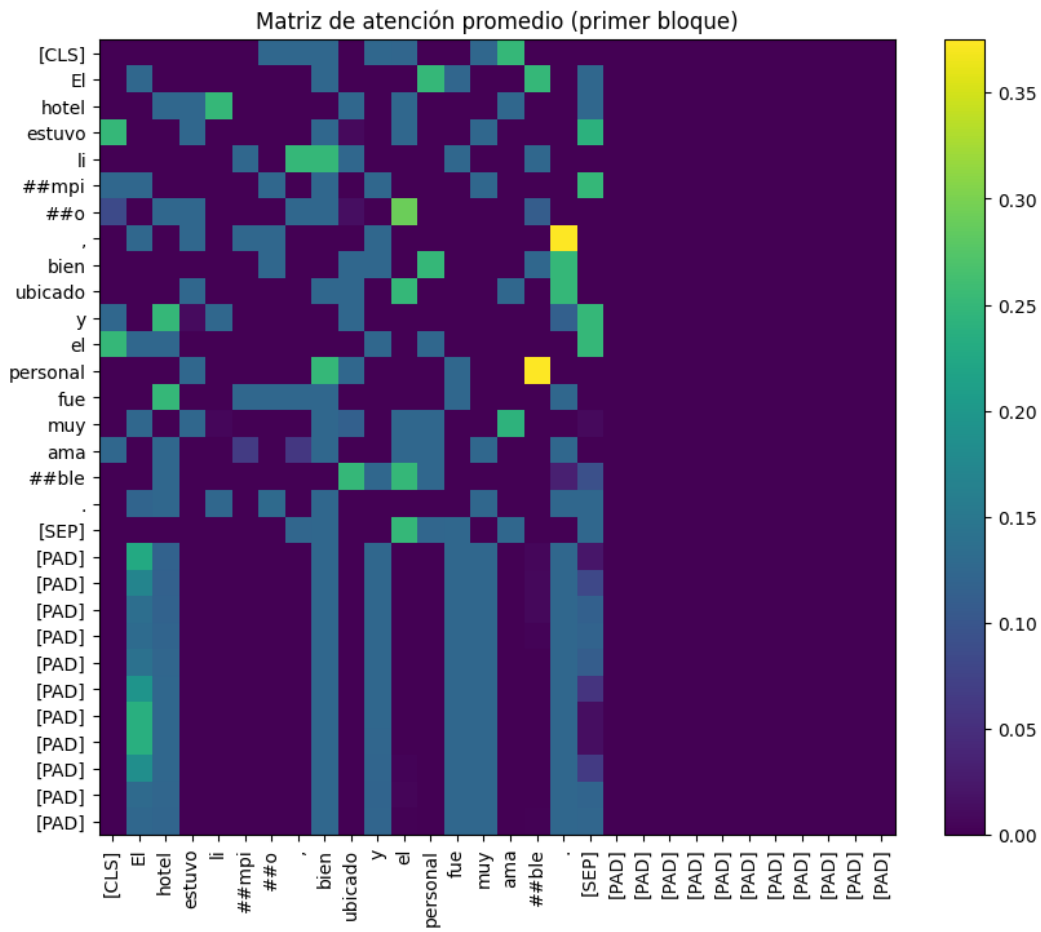
{'epoch': 3, 'train_loss': 0.6945024319864158, 'train_acc': 0.7029151561158684, 'train_macro_f1': 0.49390583941415545, 'val_loss': 0.7121882193101513, 'val_acc': 0.6952721493841876, 'val_macro_f1': 0.4732802838964634}
{'epoch': 4, 'train_loss': 0.662627078604617, 'train_acc': 0.7165650961668074, 'train_macro_f1': 0.5264620756302166, 'val_loss': 0.7132886881836962, 'val_acc': 0.6976559396106476, 'val_macro_f1': 0.4948403327891004}
{'epoch': 1, 'train_loss': 0.8590201618519868, 'train_acc': 0.6476910743297412, 'train_macro_f1': 0.3396483423453401, 'val_loss': 0.7826880123567668, 'val_acc': 0.6754072308303536, 'val_macro_f1': 0.39294910651533854}
{'epoch': 2, 'train_loss': 0.7273324013297593, 'train_acc': 0.6896270388001674, 'train_macro_f1': 0.4580238055720268, 'val_loss': 0.7247903486020768, 'val_acc': 0.6899937567398831, 'val_macro_f1': 0.45605501043219243}
{'epoch': 3, 'train_loss': 0.6916277637546638, 'train_acc': 0.7028229267912, 'train_macro_f1': 0.497008214838102, 'val_loss': 0.7152252638080375, 'val_acc': 0.6953289062943413, 'val_macro_f1': 0.4721486640715483}

8. Gráficas del Entrenamiento

Las siguientes gráficas corresponden a las métricas generadas durante el entrenamiento del modelo.




Curva de Macro F1 del mejor modelo
train_macro_f1
val_macro_f1
Macro F1
Epoch
0.550
0.525
0.500
0.475
0.450
0.425
0.400
0.375
1.0
1.5
2.0
2.5
3.0
3.5
4.0


Matriz de atención promedio (primer bloque)
[CLS]
El
hotel
estuvo
li
##mpi
##o
,
bien
ubicado
y
el
personal
fue
muy
ama
##ble
.
[SEP]
[PAD]
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00

9. Interpretabilidad del modelo

Para analizar el comportamiento del modelo se implementó una función de inspección de atención denominada inspect_attention(). Esta función permite visualizar la matriz de atención generada por el primer bloque del Transformer.

10. Discusión de resultados

Los experimentos muestran que los Transformers son capaces de capturar relaciones semánticas complejas dentro del texto. Las variaciones en el tamaño del embedding y en la codificación posicional tienen un impacto significativo en el desempeño.

11. Conclusiones

El modelo Transformer implementado logra realizar clasificación de sentimientos en reseñas turísticas con resultados competitivos. Además, la visualización de atención proporciona interpretabilidad al modelo.

12. Trabajo futuro

Como trabajo futuro se propone evaluar modelos preentrenados como BERT o RoBERTa para mejorar el desempeño del sistema.

13. Referencias

[1] A. Vaswani et al., Attention Is All You Need, NeurIPS, 2017.
[2] T. Wolf et al., Transformers: State-of-the-Art NLP, EMNLP, 2020.
[3] J. Devlin et al., BERT: Pre-training of Deep Bidirectional Transformers, NAACL, 2019.